ST JOSEPH ENGINEERING COLLEGE, MANGALORE-575028
An Autonomous Institution
Affiliated to VTU Belagavi, Recognised by AICTE New Delhi, Accredited by NAAC with A+ Grade
DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING
(UG PROGRAMME ACCREDITED BY NATIONAL BOARD OF ACCREDITATION, NEW DELHI)

Digital Design Using Verilog HDL (22ECE24)

2024-2025

IV SEMESTER B.E

Course Project Report

Submitted By

SI. No	Name of the Student	USN
1	**A V Lavanya**	**4SO23EC001**
2	**Frenny Chrystal Saldanha**	**4SO23EC028**
3	**Joel Joshua R**	**4SO23EC039**
4	**Nakul Raj E**	**4SO23EC064**

Verified By:

Dr Jennifer C Saldanha & Ms Florence Nishmitha

(Name and Signature of the Course Coordinators)

Problem Statement:

Develop a Verilog-based system that automatically regulates the speed of a fan based on temperature variations. The system should efficiently adjust the fan speed to optimize performance and energy consumption.

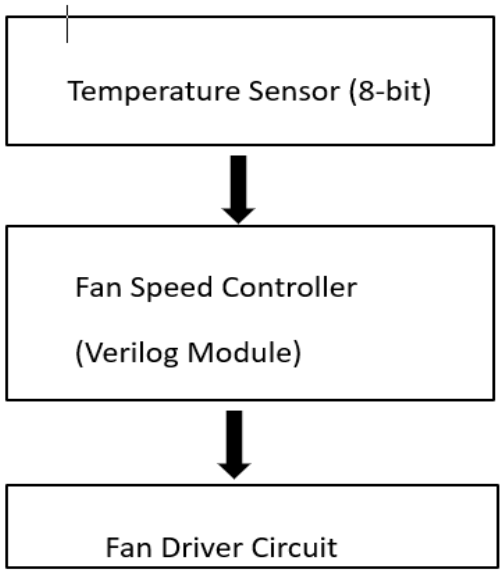
Software Used: Xilinx Vivado 2018.2

Implementation Methodology :

1. Block Diagram

The system consists of several main components:

- **Temperature Sensor Input:** Provides an 8-bit temperature reading.
- **Fan Speed Controller Module:** Implements logic to adjust fan speed based on temperature levels.
- **Clock & Reset Module:** Synchronizes the system and ensures proper initialization.
- **Fan Driver Circuit:** Receives speed signals and controls the fan motor accordingly.



2.Flow Chart

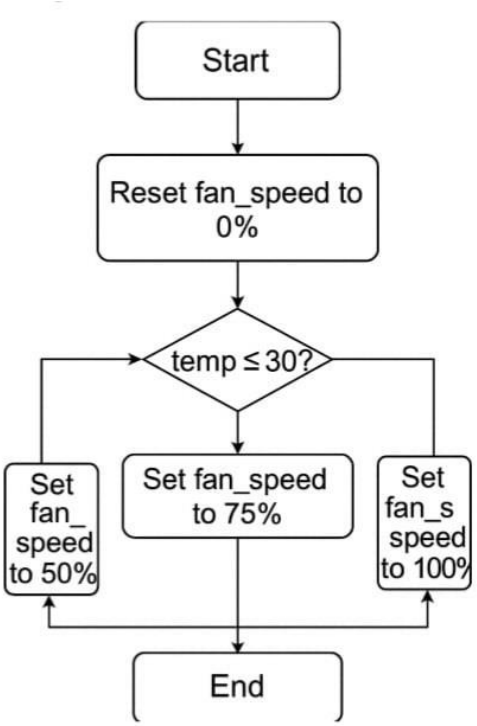
The Fan Speed Controller module operates by adjusting the fan speed based on the input temperature. Here's how it works:

1. **Initialization**: The system starts by setting the fan speed to 0% if the reset (rst) signal is active.

2. **Temperature Reading:** Once reset is deactivated, the module continuously reads the temperature input.

3. **Speed Adjustment**: Based on predefined temperature thresholds the system checks if the temperature is above certain limit**.** If the temperature is below the limit ,the fan remains at low speed and if it above the limit,the spedd is also increased.

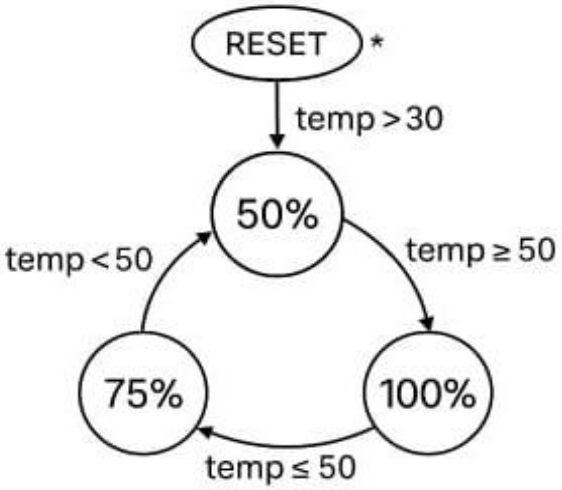
4. **Continuous Monitoring:** The process repeats, adjusting the fan speed dynamically as the temperature changes.



3.FSM Diagram Explanation

The FSM (Finite State Machine) diagram visually represents the logic behind controlling the fan speed based on temperature thresholds. The implementation follows a structured approach where temperature conditions define transitions between different fan speed states.

1. **RESET State:** Initially, the fan starts at 0% speed when the reset signal is active.
2. **50% State:** Once the reset is deasserted, the FSM transitions to a fan speed of 50% if the temperature is ≤ 30°C.
3. **75% State:** If the temperature rises above 30°C but stays ≤ 50°C, the FSM moves to this intermediate state, increasing the fan speed to 75%.
4. **100% State:** When the temperature exceeds 50°C, the FSM transitions to the final state, where the fan runs at full speed (100%).



Key Concepts from the Course, used for Implementation of the project:

1. Digital Design Principles

- **Combinational & Sequential Logic**: The system makes decisions based on temperature values (combinational logic) and updates speed at clock edges (sequential logic).
- **Finite State Machines (FSMs)**: The fan speed transitions between predefined states based on temperature thresholds.
- **Synchronization Using Clocks**: The design relies on a clock signal to ensure reliable operation.

2. Verilog Hardware Description Language (HDL)

- **Registers & Wires**: reg is used for output (fan_speed), while wire is used for connections between modules.
- **Procedural Blocks**:
    - always @(posedge clk or posedge rst): Ensures state transitions occur on clock edges.
- **Blocking vs Non-Blocking Assignments**:
    - <= (non-blocking) ensures state updates don't interfere with sequential execution.

3. Sensor-Based Control System Concepts

- **Threshold-Based Decision Making**:
    - Mapping temperature values to speed levels enhances energy efficiency.

4. Testbench Development for Verification

- **Clock Generation**: Simulating real-world behavior with a **100 MHz clock** using always #5 clk = ~clk;
- **Simulation & Monitoring**: $monitor is used to observe temperature and fan speed changes.

Verilog Code with Comments:

Design code

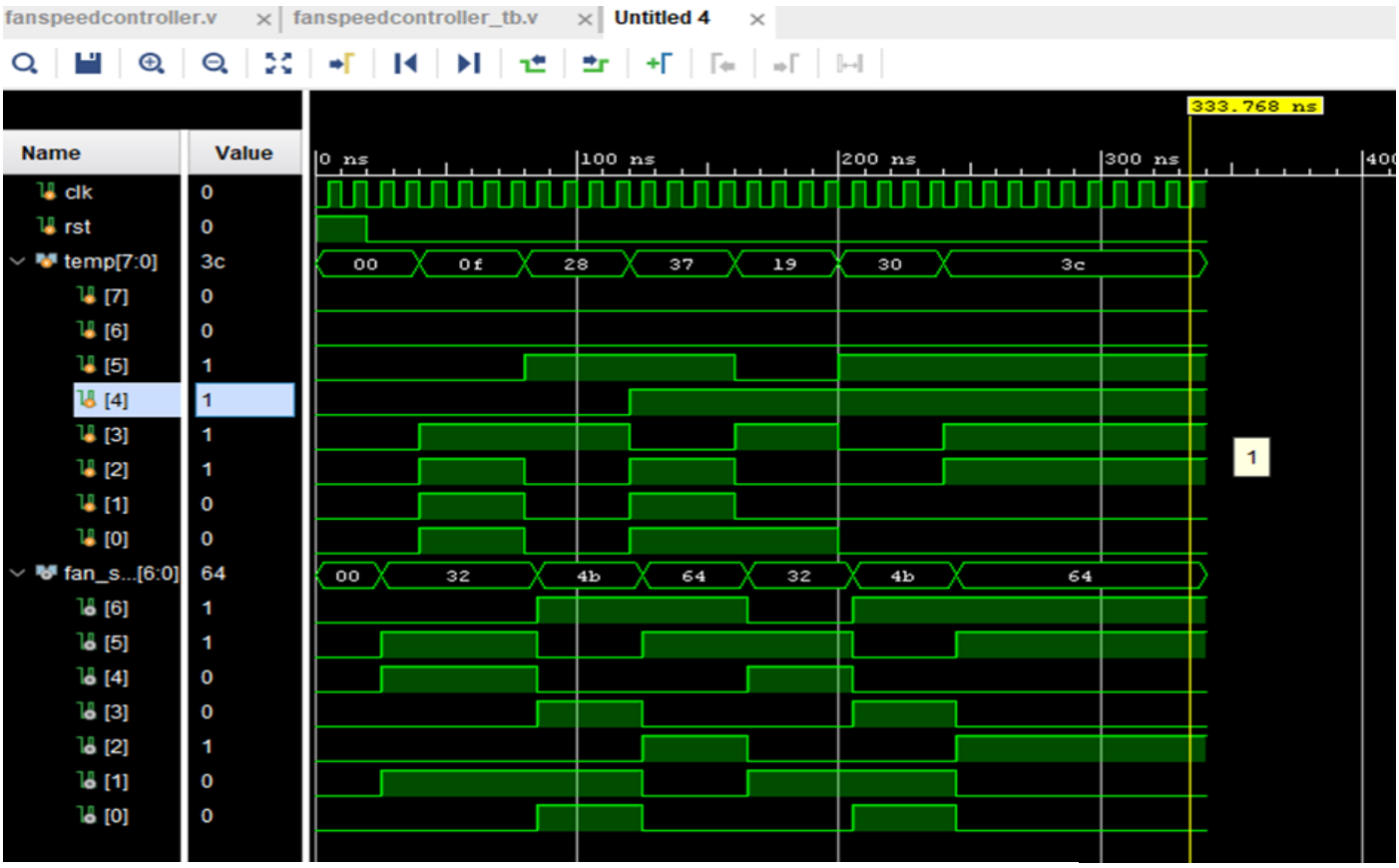
```
module fanspeedcontroller(
input wire clk,      // System clock for synchronization
input wire rst,      // Reset signal to initialize the fan speed
input wire [7:0] temp,           // Temperature input (8-bit value)
output reg [6:0] fan_speed    // Fan speed (0 to 100%)
// Fan speed control logic based on temperature thresholds
always @(posedge clk or posedge rst) begin
if (rst) begin
fan_speed <= 7'd0;               // Reset fan speed to 0% when reset signal is active
end
else begin
// Set fan speed to predefined levels based on temperature range
if (temp <= 8'd30) begin      // Temperature between 0°C and 30°C → Low fan speed (50%)
fan_speed <= 7'd50;
end else if (temp <= 8'd50) begin          // 31°C and 50°C → Medium fan speed (75%)
fan_speed <= 7'd75;
end else begin                             // Temperature above 50°C → High fan speed (100%)
fan_speed <= 7'd100;
end
end
end
endmodule
```

Testbench code

```
module fanspeedcontroller_tb();
reg clk; // Clock signal
reg rst; // Reset signal
reg [7:0] temp; // Temperature input
wire [6:0] fan_speed; // Fan speed output (percentage representation)

// Instantiate the fan speed controller module
fanspeedcontroller uut (
```

```
.clk(clk),
.rst(rst),
.temp(temp),
.fan_speed(fan_speed)
);
// Generate a clock signal with a 10 ns period (100 MHz)
always #5 clk = ~clk;

// Test sequence to verify functionality
initial begin
$monitor("Time = %0t ns | Temp = %0d | Fan Speed = %0d%%",
$time, temp, fan_speed);          // Monitor changes in values

// Initialize signals
clk = 0;
rst = 1; // Activate reset
temp = 8'd0;    // Set temperature to 0
#20 rst = 0;     // Deassert reset
// Apply different temperature values to test behavior
#20 temp = 8'd15;       // Expected fan speed: 50%
#40 temp = 8'd40;       // Expected fan speed: 75%
#40 temp = 8'd55;       // Expected fan speed: 100%
#40 temp = 8'd25;        // Expected fan speed: 50%
#40 temp = 8'd48;         // Expected fan speed: 75%
#40 temp = 8'd60;         // Expected fan speed: 100%
#100 $stop;                  // End simulation
end
endmodule
```

Snapshots of the Output:




```
# run 1000ns
Time = 0 ns | Temp = 0 | Fan Speed = 0%
Time = 25000 ns | Temp = 0 | Fan Speed = 50%
Time = 40000 ns | Temp = 15 | Fan Speed = 50%
Time = 80000 ns | Temp = 40 | Fan Speed = 50%
Time = 85000 ns | Temp = 40 | Fan Speed = 75%
Time = 120000 ns | Temp = 55 | Fan Speed = 75%
Time = 125000 ns | Temp = 55 | Fan Speed = 100%
Time = 160000 ns | Temp = 25 | Fan Speed = 100%
Time = 165000 ns | Temp = 25 | Fan Speed = 50%
Time = 200000 ns | Temp = 48 | Fan Speed = 50%
Time = 205000 ns | Temp = 48 | Fan Speed = 75%
Time = 240000 ns | Temp = 60 | Fan Speed = 75%
Time = 245000 ns | Temp = 60 | Fan Speed = 100%
INFO: [USF-XSim-96] XSim completed. Design snapshot 'fanspeedcontroller_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

Conclusion:

In this project, we designed and implemented a Verilog-based system that automatically controls the speed of a fan based on temperature. By using key digital design concepts like sequential logic, conditional statements, and testbench simulation, the system was able to respond well to different temperature levels while also improving performance and saving energy